

# BÀI LÀM – SOFTWARE ARCHITECTURE: CRYPTO STRATEGY LAB

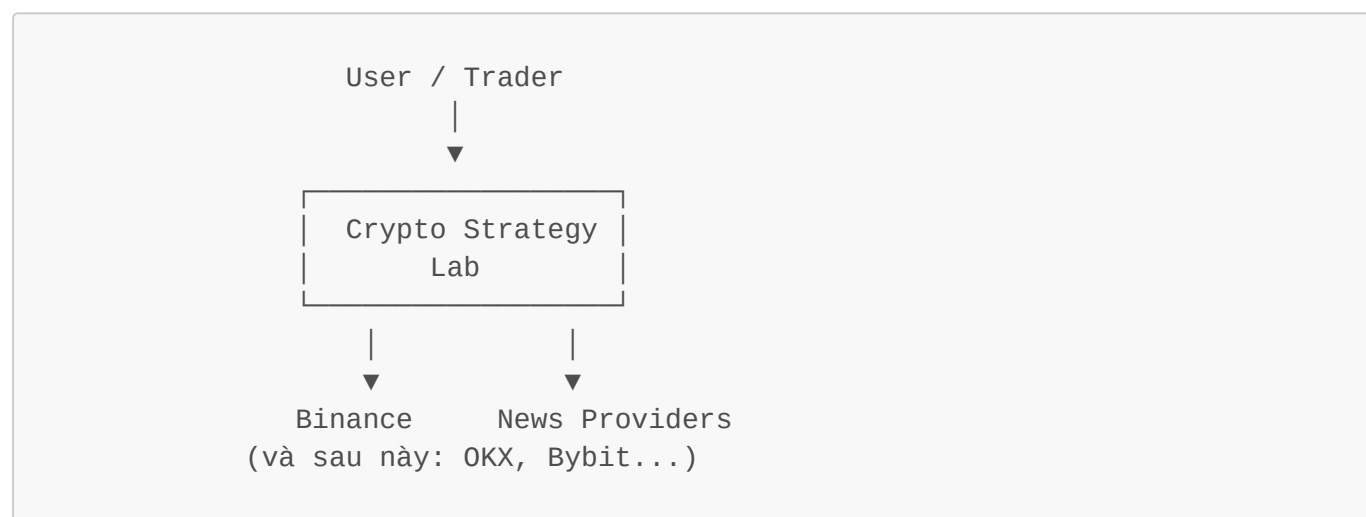
Họ và tên: Lê Thiên Phú

MSSV: 23127244

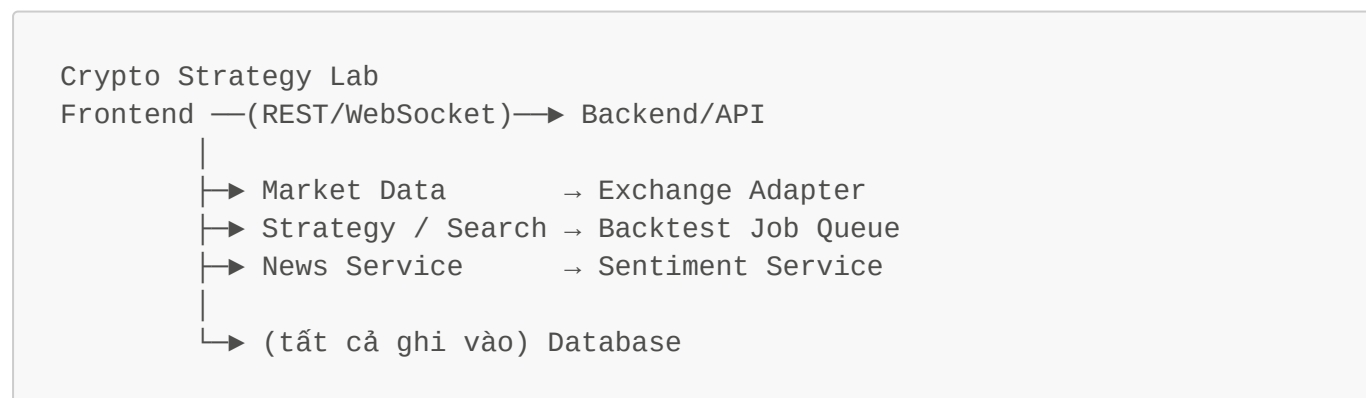
## 0. Kiến trúc nền tảng

Trước khi trả lời từng câu, em xác định một **kiến trúc ứng viên (candidate architecture)** dùng nhất quán cho cả 4 câu — vì đề bài yêu cầu giải thích "*phần nào nên thay đổi, phần nào không nên thay đổi*", nên cần một bộ khung cố định để đối chiếu qua các tình huống.

### C4 Level 1 — System Context



### C4 Level 2 — Container



Đây là **candidate architecture**, không phải đáp án duy nhất — nhưng nó sẽ là hệ quy chiếu để trả lời 4 câu bên dưới cho nhất quán.

## Câu 1 – Chia hệ thống như thế nào? (2 điểm)

### 1.1 Vì sao "một TradingService làm tất cả" là vấn đề

Bạn đề xuất gộp `getBinanceData`, `calculateMA`, `calculateRSI`, `crawlNews`, `analyzeSentiment`, `backtest`, `rank`, `saveDatabase`, `sendWebSocket` vào một class `TradingService`. Đây gọi là **God Service anti-pattern** (nguồn: P544, R2, R10): **không sai về chức năng** — hệ thống vẫn chạy được — nhưng **nguy hiểm khi phải thay đổi**, vì mọi lý do thay đổi trên đời đều dồn vào đúng một chỗ.

Cụ thể, có ít nhất 5 lý do thay đổi hoàn toàn độc lập nhau nhưng đều đụng vào cùng một class:

| Lý do thay đổi      | Nguồn gốc        | Ai là người yêu cầu       |
|---------------------|------------------|---------------------------|
| Binance đổi API     | Data provider    | Bên thứ 3 (sàn giao dịch) |
| RSI đổi công thức   | Logic chiến lược | Chuyên gia trading        |
| DB đổi schema       | Lưu trữ dữ liệu  | Team backend/DBA          |
| UI realtime đổi     | Hiển thị         | Team frontend             |
| Sentiment model đổi | Machine learning | Team ML                   |

Khi 5 đội khác nhau cùng sửa 1 file, xác suất xung đột code (merge conflict), side-effect ngoài ý muốn, và khó test tăng vọt. Đây chính là **nguyên tắc "một lý do để thay đổi"** (Single Responsibility) mà kiến trúc cần đảm bảo.

1.2 Nguyên tắc chia: "một nhà hàng không để đầu bếp kiêm thu ngân, shipper và kế toán"

Mỗi vai trò trong một nhà hàng chỉ có **một** lý do để thay đổi cách làm việc:

|                      |                              |
|----------------------|------------------------------|
| Chef (nấu ăn)        | → Market Data                |
| Cashier (thanh toán) | → Strategy                   |
| Waiter (phục vụ)     | → Backtest / Evaluate / Rank |
| Accountant (kế toán) | → UI / Presentation          |

Áp dụng vào Crypto Strategy Lab: **MarketData ≠ Strategy ≠ Backtest ≠ Evaluate ≠ Rank ≠ UI**. Nếu bếp kiêm luôn thu ngân, đổi máy tính tiền cũng phải nghĩ nấu để học lại — dù món ăn chẳng liên quan gì đến máy tính tiền. Đó chính là điều đang xảy ra với `TradingService`.

1.3 Đề xuất chia module/component

| Module                                           | Trách nhiệm                              | Lý do duy nhất để thay đổi         |
|--------------------------------------------------|------------------------------------------|------------------------------------|
| <b>MarketDataService</b> (+ Exchange Adapter)    | Lấy & chuẩn hóa dữ liệu từ Binance       | Sàn giao dịch đổi API/thêm sàn mới |
| <b>StrategyEngine</b> (Registry + Strategy impl) | Tính MA/RSI/Bollinger/SR → sinh Signal   | Thêm/sửa công thức chiến lược      |
| <b>SearchEngine / StrategyGenerator</b>          | Sinh candidate (Random/Genetic/AI Agent) | Đổi thuật toán tìm kiếm            |

| Module                           | Trách nhiệm                               | Lý do duy nhất để thay đổi     |
|----------------------------------|-------------------------------------------|--------------------------------|
| BacktestService                  | Chạy backtest cho 1 candidate             | Đổi cách mô phỏng giao dịch    |
| EvaluatorService                 | Tính metric (Return, MDD, Trades...)      | Đổi công thức đánh giá         |
| LeaderboardService               | Xếp hạng, phục vụ đọc                     | Đổi tiêu chí/định dạng ranking |
| NewsService + SentimentService   | Thu thập tin tức + phân tích cảm xúc (ML) | Đổi nguồn tin / đổi model ML   |
| Database (qua RepositoryAdapter) | Lưu trữ                                   | Đổi công nghệ lưu trữ          |
| Frontend/API                     | Hiển thị, WebSocket realtime              | Đổi UI/UX                      |

Ranh giới giữa các module không tùy tiện mà dựa trên **Domain-Driven Design — 4 Bounded Context** (R20, R21):

Market Data

→ Candle, Pair, Timeframe

Strategy

→ StrategyDefinition, Signal, Combination

Experiment

→ Candidate, Backtest, Evaluation, Ranking

News Intelligence

→ NewsItem, Sentiment

Điểm quan trọng: **Signal** trong Strategy Context **không nhất thiết là cùng khái niệm** với **Trade** trong Experiment Context — tránh nhét mọi khái niệm vào một "domain mơ hồ" tên là "Trading".

1.4 Kết nối giữa các module: đảo hướng phụ thuộc (Clean Architecture)

(R2) — thay vì để business logic gọi thẳng hạ tầng cụ thể:

✗ Không nên:

RSIStrategy → MySQL → Binance JSON

(business logic tự gọi thẳng hạ tầng cụ thể)

✓ Nên:

RSIStrategy → MarketContext/Port

→ BinanceAdapter | RepositoryAdapter

(đảo hướng phụ thuộc qua Port)

**StrategyEngine** chỉ biết một **Port** (giao diện trừu tượng), không biết đằng sau là Binance hay OKX, MySQL hay Postgres. Tương tự, Frontend chỉ nói **một ngôn ngữ dữ liệu chuẩn** (**Candle { symbol, timeframe, openTime, open, high, low, close, volume }**) thay vì "học tiếng Binance" trực tiếp.

1.5 Kiểm chứng bằng câu hỏi gợi ý của đề: "Nếu Binance đổi API, phần nào thay đổi?"

- **Thay đổi:** chỉ **BinanceAdapter** (implementation của Port **MarketDataProvider**) — vì đây là nơi duy nhất "biết" định dạng JSON của Binance.
- **Không thay đổi:** **StrategyEngine**, **BacktestService**, **EvaluatorService**, **LeaderboardService**, **Frontend** — vì tất cả chỉ phụ thuộc vào hợp đồng **Candle** chuẩn hóa, không phụ thuộc vào Binance cụ thể.

Nếu vẫn dùng **TradingService** gộp chung, đổi API Binance buộc phải mở lại **toàn bộ file**, kiểm thử lại toàn bộ luồng (MA, RSI, backtest, DB, WebSocket) dù chúng chẳng liên quan gì đến việc đổi API.

## 1.6 Vì sao cách chia này tốt hơn

1. **Modifiability (dễ sửa):** mỗi thay đổi chỉ chạm 1 module, đúng bản chất "một lý do để thay đổi."
2. **Testability:** test được từng module độc lập (mock Port thay vì cần Binance thật).
3. **Song song hóa nhóm làm việc:** 1 người làm Adapter, 1 người làm Strategy, 1 người làm UI — không giẫm chân nhau.
4. **Giảm blast radius:** lỗi ở NewsService không kéo sập MarketData/Backtest (chi tiết ở Câu 4).
5. **Có thể thay thế từng phần** mà không viết lại hệ thống — đúng yêu cầu ban đầu của khách hàng: "mở rộng được không cần viết lại toàn bộ hệ thống".

## Câu 2 – Ngày mai thêm MACD thì sao? (2 điểm)

### 2.1 Thiết kế: Strategy Interface + Plugin/Registry

(nguồn: P§12/§41, R9 – Design Patterns)

```
interface Strategy {
    analyze(context) -> Signal
}

StrategyRegistry:
    MA, RSI, Bollinger, SupportResistance, + MACD
    ← chỉ cần đăng ký thêm
```

Mọi strategy — cũ lẫn mới — đều implement **cùng một interface** **Strategy.analyze(context) -> Signal**. **StrategyRegistry** là nơi duy nhất "biết" có bao nhiêu loại strategy đang tồn tại; các module khác (Backtester, Evaluator, Leaderboard) **không biết và không cần biết** danh sách này.

Để thêm MACD:

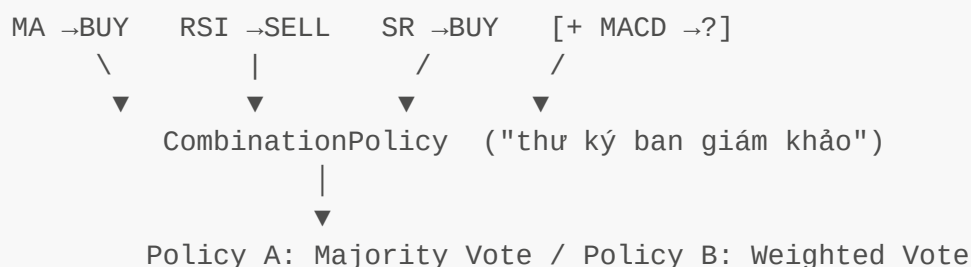
```
+ class MACDStrategy implements Strategy {
+     analyze(context) -> Signal
+ }
+ StrategyRegistry.register(MACDStrategy)
```

→ **Đúng 2 dòng thay đổi**, không đụng vào **Backtester**, **Evaluator**, **Leaderboard**, hay **Frontend core**.

## 2.2 Tách riêng "tổng hợp tín hiệu" khỏi "sinh tín hiệu"

Khi có nhiều strategy chạy song song (MA → BUY, RSI → SELL, SR → BUY), ai là người quyết định tín hiệu cuối cùng? Nếu để logic tổng hợp (**Majority Vote**, **Weighted Vote**) nằm rải rác trong từng **Strategy**, thêm MACD sẽ buộc phải sửa luôn cả logic tổng hợp.

Giải pháp: tách hẳn thành một component riêng — **CombinationPolicy** — đóng vai "thư ký ban giám khảo": mỗi strategy chỉ "chấm điểm" (**Signal**), **CombinationPolicy** mới là nơi tổng hợp:



Nguyên tắc: **Strategy signals ≠ CombinationPolicy** — thêm MACD chỉ thêm một "giám khảo" mới, không đụng vào quy trình tổng hợp điểm.

## 2.3 Vì sao Backtester/Evaluator/Leaderboard không cần sửa — bản chất nằm ở đâu?

Đây chính là điều đề bài hỏi: "điều gì trong thiết kế giúp thêm strategy mới mà ít ảnh hưởng code cũ?" Bản chất (không cần nhớ tên pattern, theo đúng lưu ý của đề) là:

1. **Lập trình theo hợp đồng (interface/contract), không lập trình theo từng lớp cụ thể.** **Backtester** chỉ gọi **CandidateStrategy.analyze()** → **Signal**, nó không quan tâm bên trong là MA hay MACD.
2. **Dependency Inversion:** các module downstream (Backtester, Evaluator, Leaderboard) phụ thuộc vào **abstraction** (**Strategy**, **CandidateStrategy**), còn các strategy cụ thể mới là thứ phụ thuộc ngược lại vào abstraction đó — không phải chiều ngược lại.
3. **Open/Closed Principle:** hệ thống "mở" cho việc mở rộng (thêm class mới) nhưng "đóng" với việc sửa đổi (không cần sửa code cũ).
4. **Registry là điểm biến thiên (variation point) duy nhất được thiết kế sẵn** — mọi "biến động" tương lai (thêm/bớt strategy) chỉ xảy ra ở đây.

## 2.4 Kiểm chứng — "Architecture Extensibility Test" (Architecture Proof #1)

|                      | Thiết kế tốt (Plugin + Registry)            | Thiết kế kém (coupling cao)                                                                                       |
|----------------------|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Nơi phải sửa         | + <b>MACDStrategy</b> , + <b>register()</b> | <b>Controller</b> , <b>Backtester</b> , <b>UI</b> , <b>Database</b> , <b>CombinationEngine</b> , <b>Evaluator</b> |
| Số dòng/nơi thay đổi | 2 dòng                                      | 6 nơi                                                                                                             |
| Ý nghĩa              | Thay đổi thật là "unit test" của kiến trúc  | Mọi thay đổi nhỏ đều lan rộng → kiến trúc mong manh                                                               |

→ Đây cũng chính là câu trả lời sống động cho **Câu 4 – Scenario A** nếu được chọn: thử nghiệm "thêm MACD" và đếm số nơi phải sửa là cách đo được (measurable) độ tốt của kiến trúc, không phải cảm tính "nhìn gọn."

## Câu 3 – Từ 100 lên 100.000 backtests (3 điểm)

### 3.1 Vì sao 100.000 không phải một con số ngẫu nhiên

Ban đầu 4 strategy (MA, RSI, Bollinger, SR) đã "vui." Nhưng mỗi strategy có nhiều tham số (MA windows: 10/20/50/200..., RSI thresholds: 14/30/70, 20/80..., BB deviations, SR detection params...). Đây là **bùng nổ tổ hợp (combinatorial explosion)**: giống chọn trang phục — 5 áo × 4 quần × 3 giày = 60 tổ hợp; với strategy trading, vài tham số nhỏ nhân nhau tạo ra **hàng chục nghìn đến hàng trăm nghìn candidate** — 100.000 là hệ quả tất yếu của phép nhân tổ hợp, không phải số bịa.

### 3.2 Kiến trúc mới: từ for-loop sang Job Queue + Workers

**Vấn đề với kiến trúc cũ:**

```
Kém scalable:
for candidate in candidates:
    backtest(); evaluate(); update_ui()
```

Một for-loop tuần tự = "một quầy thu ngân duy nhất phục vụ 10.000 khách" → với 10.000 candidate, ước tính ~5.5 giờ chạy tuần tự (minh họa, không phải cam kết hiệu năng).

**Kiến trúc đề xuất** — tách pipeline thành sinh việc / xử lý việc song song:

```
StrategyGenerator
  |
  ▼
Job Queue (BacktestJob)
  |
  ▼
Workers (1, 2, 3, ...) ← chạy song song
  |
  ▼
Evaluator → Ranking / Leaderboard
```

Mỗi **BacktestJob** là một đơn vị công việc độc lập (một candidate cụ thể). Nhiều Worker lấy job từ Queue và xử lý **song song** thay vì "một người làm hết". Với 3 worker, thời gian ước tính giảm còn ~1.8 giờ — càng nhiều worker càng nhanh (tuyến tính, với điều kiện worker không tranh chấp tài nguyên).

Ngoài **parallelism**, kiến trúc Queue + Workers còn "mua được": **retry, pause/resume, backpressure, observability** — những thứ một for-loop đơn giản không có.

Kết quả (**BacktestCompleted**) được **publish qua event** thay vì gọi hàm trực tiếp **LeaderboardService.update()** — để Worker không cần biết có bao nhiêu "người nghe" (Ranking, Audit...) đang tồn tại; thêm consumer mới = "đăng ký lắng nghe", Worker không đổi gì cả.

Vì search có thể sinh candidate liên tục, bắt buộc phải có **Stop Condition** (`max candidates`, `max time`, `no improvement N iterations`, `user cancel`) — **không** được viết `while(true)`.

### 3.3a. Nếu từ 1 worker tăng thành 4 worker — phần nào KHÔNG nên phải sửa?

Đây chính là phép thử "Scalability Test" của kiến trúc (Architecture Proof #3):

#### Không nên sửa:

- **StrategyGenerator / SearchCoordinator** — vẫn sinh candidate như cũ, không biết có bao nhiêu worker đang nghe.
- **Hợp đồng BacktestJob** (schema của một job) — worker mới chỉ cần đọc đúng cấu trúc job đã định nghĩa sẵn.
- **EvaluatorService, LeaderboardService** — vẫn nhận **BacktestCompleted** như cũ, không quan tâm job đó được worker nào xử lý.
- **Code nghiệp vụ của chính Worker** (logic backtest) — không đổi, chỉ đổi *số lượng bản sao (replicas)* đang chạy.

**Chỉ cần đổi:** cấu hình scale (số replicas: **1** → **4**, giống **Docker** đóng gói 1 image chạy nhiều container, **Kubernetes** chỉ tăng **replicas**).

**Điều kiện bắt buộc để việc scale "miễn phí" này đúng: Worker phải stateless và Job phải idempotent** — nếu không, thêm worker chỉ tạo ra nhiều lỗi hơn nhanh hơn (ví dụ: 2 worker cùng ghi đè một candidate, hoặc worker giữ trạng thái cục bộ khiến kết quả phụ thuộc vào việc job được chia cho worker nào).

### 3.3b. Nếu một worker bị lỗi giữa lúc backtest — xử lý job đó thế nào?

**Bản chất vấn đề — Transaction boundary** (*R11 – Principles of Transaction Processing*): một "kết quả backtest hoàn tất" gồm 4 bước — (1) lưu trades, (2) lưu metrics, (3) đánh dấu **COMPLETED**, (4) publish event **BacktestCompleted**. Nếu worker crash **sau bước 2**: trades ✓, metrics ✓, nhưng status **X** và event **X** chưa publish → **Leaderboard đọc dữ liệu không nhất quán** (thấy trades/metrics nhưng chưa bao giờ được thông báo).

**Tư duy đúng — giống giao dịch ATM:** rút tiền ATM là trừ tài khoản A và cộng tài khoản B — **hoặc cả hai cùng thành công, hoặc cùng bị hủy (rollback)**, không có trạng thái lửng lơ ở giữa. Backtest worker cần tư duy y hệt: **Trades + Metrics + Status + Event** phải là **một đơn vị toàn vẹn duy nhất (atomic)**.

#### Cơ chế xử lý cụ thể khi worker lỗi giữa chừng:

1. **Atomicity ở phạm vi cần thiết:** gộp (trades, metrics, status) trong một transaction DB; publish event **sau khi** transaction commit thành công (outbox pattern / "commit rồi mới báo tin", tránh publish event cho dữ liệu chưa chắc đã lưu).
2. **Phát hiện job "treo":** dùng heartbeat/timeout — nếu worker không báo cáo tiến độ trong khoảng thời gian quy định, Job Queue coi job đó là thất bại và **đưa job trở lại hàng đợi (requeue)**.
3. **Retry có giới hạn + Idempotency key:** mỗi job có một khóa duy nhất; khi retry, hệ thống kiểm tra khóa này để **không tạo bản ghi trùng (duplicate)** nếu job thực chất đã hoàn tất một phần trước khi crash. Sau N lần retry thất bại, job được đánh dấu **FAILED** (dead-letter) thay vì retry vô hạn.
4. **Cô lập lỗi (fault isolation):** vì mỗi job độc lập và mỗi worker chỉ xử lý job của riêng nó, một worker chết **không ảnh hưởng** đến các job đang chạy trên các worker khác — đây là lợi ích trực tiếp của kiến trúc Queue + Workers so với for-loop tuần tự (nếu for-loop lỗi giữa chừng, toàn bộ tiến trình dừng).

5. **Observability:** hệ thống cần biết được có bao nhiêu job đang chạy, bao nhiêu lỗi, backlog dài bao nhiêu — đúng driver "Observability" đã đặt ra ngay từ đầu.

## Câu 4 – Kiến trúc của bạn có thật sự tốt không? (3 điểm)

### 4.0 Vì sao "sơ đồ đẹp" không chứng minh được gì

(ATAM — *Architecture Evaluation, R1, W8*): một sơ đồ hộp và mũi tên chỉ là **lời tuyên bố**. Cách duy nhất để biết kiến trúc thật sự tốt là "**crash-test**" nó bằng các tình huống thay đổi/thất bại cụ thể và quan sát hệ thống phản ứng ra sao — đây chính là tư tưởng của **ATAM (Architecture Tradeoff Analysis Method)**.

Em chọn 2 tình huống: **B – Random Search → Genetic Search** và **C – News Service bị tắt**, vì hai tình huống này **chưa được phân tích sâu ở Câu 2 và Câu 3**, giúp kiểm chứng kiến trúc trên các thuộc tính chất lượng khác: *Replaceability* (khả năng thay thế thuật toán) và *Reliability/Fault isolation* (khả năng cô lập lỗi).

### 4.1 Scenario B — Random Search → Genetic Search (Architecture Proof #2 — Replaceability Test)

**Thiết kế liên quan:** `StrategyGenerator` là một interface (Port): `interface StrategyGenerator { generate() -> CandidateStrategy }`. Hiện tại implementation là `RandomStrategyGenerator`; yêu cầu mới là `DomainGuidedStrategyGenerator` (hoặc `GeneticStrategyGenerator`) — **hợp đồng (contract) không đổi**.

**1. Bạn sẽ thử điều gì?** Viết một implementation mới của `StrategyGenerator` dùng thuật toán di truyền (chọn lọc, lai ghép, đột biến trên tập candidate), rồi **hoán đổi implementation đang được inject vào `SearchCoordinator`** — không sửa bất kỳ dòng nào ở `BacktestService`, `EvaluatorService`, `LeaderboardService`.

### 2. Kết quả nào chứng minh kiến trúc tốt?

- `Backtester`, `Evaluator`, `Leaderboard` **không cần sửa một dòng code nào** — vì chúng chỉ nhận `CandidateStrategy` qua Queue, không quan tâm nó được sinh ra bằng thuật toán ngẫu nhiên hay di truyền.
- Chỉ cần đổi **1 chỗ duy nhất** (nơi khởi tạo/inject `StrategyGenerator`), tương tự cách MACD chỉ cần đăng ký ở Registry (Câu 2).
- Hệ thống vẫn publish đúng các event (`StrategyGenerated`, `BacktestCompleted...`) theo đúng schema cũ.

### 3. Nếu điều gì xảy ra thì kết luận kiến trúc đang có vấn đề?

- Phải sửa `Evaluator` hoặc `Leaderboard` để "hiểu" cách Genetic Search sinh ra candidate (tức là chúng đang phụ thuộc vào cách sinh ra dữ liệu, không phải vào contract).
- `BacktestJob` phải thêm field đặc thù cho thuật toán generator (rõ ràng chi tiết implementation ra ngoài interface).
- Kết quả (fitness/score dùng để chọn lọc thế hệ tiếp theo trong Genetic Search) không truy vết được về đúng generation/version — vi phạm Reproducibility đã đặt ra ở driver ban đầu.



## 4.2 Scenario C — News Service bị tắt (Architecture Proof #3 — phần Failure Test)

**Thiết kế liên quan:** `NewsService/SentimentService` là một Bounded Context tách biệt, giao tiếp với phần còn lại của hệ thống qua **event** (`NewsCollected`, `SentimentAnalyzed`) chứ không phải lời gọi đồng bộ nằm trên đường đi chính (critical path) của Market Data/Strategy/Backtest.

**1. Bạn sẽ thử điều gì?** Chủ động tắt (kill) `NewsService/SentimentService` trong lúc hệ thống đang chạy, rồi quan sát: Realtime Chart còn cập nhật không? `StrategyEngine` (MA/RSI/Bollinger/SR — các strategy không phụ thuộc sentiment) còn tính được Signal không? `BacktestService` còn chạy được không?

### 2. Kết quả nào chứng minh kiến trúc tốt?

- **Realtime Chart ✓, Strategy Engine ✓, Backtest ✓** vẫn hoạt động bình thường — vì `MarketDataService` không phụ thuộc vào `NewsService`.
- **Chỉ** phần News/Sentiment trên UI bị "degraded" (hiển thị dữ liệu cũ hoặc trống) — đúng như driver mini-challenge D đã đặt ra ngay từ đầu: *"News Service lỗi không làm Market Data pipeline ngừng"*.
- Nếu hệ thống có dùng `SentimentStrategy` (composite kèm tín hiệu tin tức), `CombinationPolicy` vẫn tổng hợp được tín hiệu từ các strategy còn lại (MA/RSI/...) — không throw lỗi toàn cục chỉ vì thiếu 1 input.

### 3. Nếu điều gì xảy ra thì kết luận kiến trúc đang có vấn đề?

- Toàn bộ hệ thống (kể cả Market Data, Backtest, Leaderboard) **cũng dừng/crash theo** — dấu hiệu rõ ràng cho thấy các module đang bị **coupling chặt** (ví dụ: gọi đồng bộ, blocking call tới News Service nằm trên đường đi chính) thay vì giao tiếp qua event bất đồng bộ.
- `CombinationPolicy` throw exception toàn cục vì thiếu 1 tín hiệu đầu vào, thay vì xử lý "tín hiệu vắng mặt" một cách graceful (bỏ qua / dùng giá trị mặc định).
- Không có ranh giới (boundary) rõ ràng giữa Bounded Context "News Intelligence" và các Context còn lại → đúng như bài học ở phần phân tích driver ban đầu: đây mới là loại ASR (Architecturally Significant Requirement) thật sự lái kiến trúc, khác với các yêu cầu bề mặt như "chart phải đẹp."

## 4.3 Bảng trade-off tổng hợp (tham chiếu chung cho cả 2 scenario)

| Quyết định kiến trúc                     | Lợi ích                                      | Cái giá phải trả                                      |
|------------------------------------------|----------------------------------------------|-------------------------------------------------------|
| Plugin/Registry cho Strategy & Generator | Thêm/thay thuật toán dễ, không sửa code cũ   | Phải giữ kỷ luật versioning cho contract              |
| Event-driven (News tách biệt)            | Cô lập lỗi (Fault isolation), loose coupling | Cần quản lý thứ tự event, khó trace hơn gọi trực tiếp |
| Job Queue + Workers                      | Scale, retry được                            | Eventual consistency, cần idempotency                 |

**Kết luận chung:** kiến trúc tốt không phải là kiến trúc "không có nhược điểm", mà là kiến trúc có **trade-off phù hợp với driver thật sự của bài toán** — không phải là kiến trúc dùng nhiều công nghệ phức tạp nhất.